AI Service Review and Test Report

Date: January 9, 2026
Project: PSScript Platform
Component: AI Service (`src/ai/`)

Executive Summary

The AI service has a solid architecture with FastAPI, agent coordination, and multi-model support, but contains **critical bugs** and **deprecated model references** that will prevent the service from running in production. The main issues are:

1. **Missing Import**: `agent_factory` is used but never imported in `main.py` (CRITICAL)
2. **Non-existent Models**: References to models that don't exist (`gpt-5.2-codex`, `gpt-5.2`, `gpt-5.2-instant`)
3. **Deprecated OpenAI API**: Using old SDK style instead of modern OpenAI client
4. **Incomplete Implementations**: Several placeholder methods in `AgentCoordinator`

Overall Status: ❌ **Not Ready for Production** - Critical bugs must be fixed before deployment

---

Critical Issues (Must Fix)

1. Missing `agent_factory` Import in `main.py`

Severity: 🔴 **CRITICAL**
File: `src/ai/main.py`
Lines: 407, 508, 550, 591, 1016, 1020

Issue: The code attempts to use `agent_factory.get_agent()` and `agent_factory.process_message()` but never imports the `agent_factory` module.

```
# BROKEN - in main.py (line ~407)
agent = agent_factory.get_agent("hybrid", api_key or config.api_keys.opena
# NameError: name 'agent_factory' is not defined
```

Fix Required:

```
# Add this import at the top of main.py
from agents.agent_factory import agent_factory
```

Impact: All fallback agent processing will fail, affecting: - `analyze_script_security()` endpoint - `categorize_script()` endpoint
- `find_documentation_references()` endpoint - `chat_with_powershell_expert()` endpoint

---

2. Non-existent OpenAI Model References

Severity: 🔴 **CRITICAL**
Files: Multiple
Affected Models: `gpt-5.2-codex`, `gpt-5.2`, `gpt-5.2-instant`

Issue: Configuration and code reference non-existent OpenAI models that will cause API errors.

Files with issues: - `src/ai/config.py` (lines 46-47, 134, 136) - `src/ai/analysis/script_analyzer.py` (line 55) - `src/ai/agents/langchain_agent.py` (line 67) - `src/ai/agents/langgraph_production.py` (lines 410, 473, 651, 880) - `src/ai/utils/token_counter.py` (lines 20, 24, 28, 301, 310)

Current Config (in `config.py`):

```
default_model: str = Field("gpt-5.2-codex", ...)  # INVALID
reasoning_model: str = Field("gpt-5.2", ...)     # INVALID
```

Fix Required: Use actual OpenAI models:

```
# Recommended updates for January 2026
default_model: str = Field("gpt-4-turbo", ...)     # For general tasks
analysis_model: str = Field("gpt-4-turbo", ...)    # For script analysis
embedding_model: str = Field("text-embedding-3-large", ...)  # For embeddi
reasoning_model: str = Field("gpt-4o", ...)        # For complex reasoning
```

Available OpenAI Models (as of January 2026): - **Text Models**: `gpt-4o`, `gpt-4-turbo`, `gpt-3.5-turbo` - **Embeddings**: `text-embedding-3-large`, `text-embedding-3-small` - **Vision**: `gpt-4-vision`, `gpt-4o`

---

3. Deprecated OpenAI API Usage

Severity: 🟠 **HIGH**
File: `src/ai/analysis/script_analyzer.py`
Lines: Throughout the file

Issue: Using deprecated OpenAI SDK pattern.

Current (DEPRECATED):

```
import openai
openai.api_key = os.getenv("OPENAI_API_KEY")  # Old way
response = openai.embeddings.create(...)
response = openai.chat.completions.create(...)
```

Fix Required (Modern OpenAI SDK):

```
from openai import OpenAI

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
response = client.embeddings.create(...)
response = client.chat.completions.create(...)
```

Impact: The service may fail with newer versions of the OpenAI library.

High Priority Issues (Should Fix)

4. Undefined Variable in `main.py`

Severity: 🟠 **HIGH**
File: `src/ai/main.py` (line ~49)

Issue: Reference to `agent_factory` global in chat endpoint fallback:

```
response = await agent_factory.process_message(messages, api_key)
```

Note: This is related to issue #1 above.

---

5. Missing Error Handling in Async Event Loops

Severity: 🟠 **HIGH**
Files: `src/ai/analysis/script_analyzer.py`

Issue: Creating new event loops in synchronous wrappers can cause issues:

```
def generate_embedding(self, text: str) -> List[float]:
    loop = asyncio.new_event_loop()  # Problematic pattern
    try:
        return loop.run_until_complete(...)
    finally:
        loop.close()
```

Better Approach: Use asyncio.run() or handle existing event loops:

```
def generate_embedding(self, text: str) -> List[float]:
    try:
        loop = asyncio.get_running_loop()
    except RuntimeError:
        return asyncio.run(self.generate_embedding_async(text))
    # ... handle existing loop case
```

---

6. Incomplete AgentCoordinator Methods

Severity: 🟠 **HIGH**
File: `src/ai/agents/agent_coordinator.py`

Issue: Several methods return placeholder implementations:

```
 async def search_similar_scripts(self, script_content: str, limit: int = 5
    logger.info(f"Searching for similar scripts (placeholder)")
    return []  # TODO: Implement vector similarity search

async def generate_script_embedding(self, script_content: str):
    logger.info(f"Generating script embedding (placeholder)")
    return [0.0] * 1536  # TODO: Implement actual embedding generation
```

Impact: These features won't work when calling the agent coordinator directly.

Medium Priority Issues (Nice to Fix)

7. Inconsistent Error Handling

Severity: 🟡 **MEDIUM**
File: `src/ai/analysis/script_analyzer.py`

Issue: Error responses don't maintain consistency with success responses:

```
 error_response = {
    "purpose": "Error analyzing script",
    "security_analysis": f"Analysis failed: {str(e)}",  # Different format
    "security_score": 5,  # Default value vs. actual analysis
    ...
}
```

Recommendation: Return consistent error structure:

```
if error:
    return {
        "error": {
            "code": "ANALYSIS_FAILED",
            "message": str(e),
            "timestamp": datetime.now().isoformat()
        }
    }
```

---

8. Missing Type Validation in Pydantic Models

Severity: 🟡 **MEDIUM**
File: `src/ai/main.py`

Issue: Some response models don't validate expected data:

```
class EmbeddingResponse(BaseModel):
    embedding: List[float]
```

Missing validation could return embeddings of wrong dimensions (expects 3072 for text-embedding-3-large).

Recommendation:

```
class EmbeddingResponse(BaseModel):
    embedding: List[float] = Field(..., min_items=256, max_items=3072)
    dimension: int
    model: str
```

---

9. Missing Database Error Handling

Severity: 🟡 **MEDIUM**
File: `src/ai/main.py` (lines ~440-475)

Issue: Database errors silently continue instead of propagating:

```
try:
    # Database operations...
except Exception as e:
    print(f"Database error: {e}")
    # Continue without saving
finally:
    if conn:
        conn.close()
```

Impact: Analysis results won't be saved if DB fails, but client doesn't know.

Low Priority Issues

10. Token Counter Using Non-existent Models

Severity: 🟡 **LOW**
File: `src/ai/utils/token_counter.py`

Issue: Token counting configured for non-existent models:

```
"gpt-5.2": {
    "input": 0.00150,
    "output": 0.00600
},
"gpt-5.2-codex": {
    "input": 0.00200,
    "output": 0.00800
}
```

Recommendation: Remove these and use actual model token counts.

---

11. Inconsistent Error Handling in Agent Coordinator

Severity: 🟡 **LOW**
File: `src/ai/agents/agent_coordinator.py`

Issue: Tasks timeout after 300 seconds but error isn't clearly communicated:

```
if not tasks_completed and time.time() - start_time < timeout:
    # Task timed out silently
    results["analysis"] = {"error": "Analysis task failed or timed out"}
```

Architecture & Design Notes

Strengths ✅

1. **Clean separation of concerns**: Agents, tools, memory systems separate
2. **Multi-agent system**: Good coordinator pattern for delegating tasks
3. **Caching strategy**: Disk + Redis fallback for analysis caching
4. **Type safety**: Pydantic models for request/response validation
5. **CORS enabled**: Allows frontend integration
6. **Health check endpoint**: Good for monitoring

Areas for Improvement 📝

1. **Mock mode testing**: Properly implemented for development without API keys
2. **Vector DB integration**: pgvector check is good, but implementations incomplete
3. **Agent communication**: Needs better error propagation from agents to API responses
4. **Rate limiting**: Configured but not enforced on routes
5. **Logging**: Good structure, but some logs are at wrong levels

---

Deployment Readiness Checklist

- [ ] Fix missing `agent_factory` import in main.py
- [ ] Update all model references to valid OpenAI models
- [ ] Migrate from deprecated OpenAI API to modern SDK
- [ ] Implement actual embedding generation (currently placeholder)
- [ ] Implement actual vector similarity search (currently placeholder)
- [ ] Add proper error handling for agent coordinator task timeouts
- [ ] Test all endpoints with actual API keys

- [ ] Validate database connection on startup
- [ ] Add comprehensive unit tests
- [ ] Add integration tests for agent workflows
- [ ] Document API changes for frontend team
- [ ] Set up proper rate limiting enforcement
- [ ] Configure production logging levels
- [ ] Add request/response logging for debugging

Testing Recommendations

Unit Tests Needed

1. **AgentFactory**: Test agent creation and fallback
2. **ScriptAnalyzer**: Test with mock embeddings before API calls
3. **AgentCoordinator**: Mock the multi-agent system
4. **Pydantic Models**: Validate input/output schemas

Integration Tests Needed

1. **Full script analysis flow**: From upload to results
2. **Agent coordination**: Multi-agent task execution
3. **Database integration**: Analysis storage and retrieval
4. **Error scenarios**: API failures, timeouts, invalid inputs

End-to-End Tests

1. Chat endpoint with different agent types
2. Script upload and analysis workflow
3. Vector similarity search
4. Token usage tracking and cost estimation

Recommendations

Immediate (Critical)

1. Add `agent_factory` import to `main.py` line 1

2. Update model references to valid OpenAI models
3. Update ScriptAnalyzer to use modern OpenAI SDK

Short-term (Next Sprint)

1. Implement actual embedding generation
2. Implement actual vector similarity search
3. Add comprehensive error handling for all agent tasks
4. Create unit test suite for core components

Medium-term (Next Release)

1. Add streaming response support for chat endpoint
2. Implement agent metrics and monitoring
3. Add request/response caching strategy
4. Optimize database queries for analysis storage

Files Requiring Changes

File	Issues	Priority
`src/ai/main.py`	Missing import, undefined variables	CRITICAL
`src/ai/config.py`	Invalid model names	CRITICAL
`src/ai/analysis/script_analyzer.py`	Deprecated API, async issues, incomplete implementations	CRITICAL
`src/ai/agents/agent_coordinator.py`	Placeholder implementations, timeout handling	HIGH
`src/ai/agents/langchain_agent.py`	Invalid model reference	HIGH
`src/ai/agents/langgraph_production.py`	Invalid model references	HIGH
`src/ai/utils/token_counter.py`	Invalid model pricing	MEDIUM

Conclusion

The AI service has a well-designed architecture with proper separation of concerns and sophisticated agent coordination. However, it currently **cannot be deployed to production** due to critical bugs and non-existent model references. Once the critical issues are resolved, the service will provide a robust platform for PowerShell script analysis with multi-agent support.

Estimated Time to Fix Critical Issues: 2-4 hours
Estimated Time for Full Testing Suite: 16-24 hours

Report Generated: January 9, 2026
Reviewer: Claude Code AI Review System

Generated 2026-04-29 14:08 UTC